

PROFILE

React·TypeScript로 WebView 거래 화면과 데이터 분석 UI를 개발해왔습니다. 느린 화면은 측정하고, 여러 기능이 얹힌 코드는 함께 쓸 부분과 따로 바꿀 부분을 나눠 정리합니다. 화면 개발에 필요한 BFF와 모델 조회 API도 직접 만들었고, 최근에는 사내 AI Agent의 실행 환경과 권한·리뷰 자동화를 개선하고 있습니다.

EXPERIENCE

2024.09 - 현재

Frontend Developer

2년

파이레테크놀로지

Frontend Developer

WebView 거래 제품과 BFF, 블록 스캐너 성능 개선, 사내 AI Agent 런타임 개발을 맡고 있습니다.

TypeScript

React

Next.js

TailwindCSS

TanStack Query

Jotai

NestJS

wagmi · viem

Chakra UI

Vitest

Playwright

Sentry

- 블록 스캐너의 초기 행을 단계적으로 렌더링하고 실시간 행 추가량·갱신 빈도를 제한해, 로컬 Lighthouse 기준 LCP 평균 **45%**, TBT 평균 **74%** 단축
- Pockie의 BiFi·Swap·BTCFi WebView 거래 화면과 NestJS 기반 Swap BFF 개발 — 견적·거래 데이터 생성·상태 조회 API 연동
- BTCFi Partners에서 공통 모듈 변경이 다른 파트너 화면에 영향을 주는 문제를 다루기 위해, 파트너별 거래 흐름을 분리하고 CI에 의존 규칙 검사 추가
- 사내 AI Agent의 Claude SDK 실행 경로를 Codex App Server로 전환하고, 도구별 실행 권한과 자동 코드 리뷰 검증 흐름 개선

2021.07 - 2024.09

3년 2개월

Full-stack Engineer

→ Frontend Developer

→ Frontend Part Leader

스마트마인드

Frontend Developer → Frontend Part Leader

AI 데이터 분석 플랫폼 Workspace의 SQL Editor·Query Viewer 모듈을 개발했으며, pnpm·Turborepo 모노레포 전환과 Microfrontend 설계를 주도했습니다. 후반에는 프론트엔드 파트 리더로 팀 운영을 담당했습니다.

TypeScript

React

Next.js

Vite

Monaco Editor

ANTLR

pnpm · Turborepo

Playwright

FastAPI

Docker Compose

- 늘어나는 앱을 한 저장소에서 관리하도록 pnpm·Turborepo로 묶고 빌드 캐시를 적용해, 당시 측정 기준 앱별 빌드를 **7분 → 1분**으로 단축
- 번들 분리·dynamic import로 Workspace 초기 로드 범위를 줄여 당시 측정 기준 First Load 최대 **40%** 단축
- AI 팀이 정의한 ANTLR 문법을 Monaco Editor에 연결해 SQL 구문 강조·오류 진단·키워드 자동완성 구현
- Workspace에 여러 앱을 연결하기 위해 Lab·Main은 iframe·postMessage로 연동하고 Query Manager·File Manager는 module federation으로 구성
- 대용량 쿼리 결과를 한꺼번에 그리지 않도록 서버 페이지네이션·가상 스크롤을 적용하고 이미지·영상은 필요한 시점에 로딩
- FastAPI·Docker Compose로 AI 예측 모델 서빙 환경과 데이터 수집·전처리 파이프라인 개발
- 2023년부터 본인 포함 2~3명 프론트엔드 파트의 스크럼 운영·업무 배분·우선순위 결정 담당

01 /

블록 스캐너(Explorer) 성능 최적화

Route	LCP (before → after)	TBT (before → after)	개선
/blocks	3.7s 1.9s	1,010ms 270ms	LCP ↓49% · TBT ↓73%
/txs	3.7s 2.2s	1,330ms 200ms	LCP ↓41% · TBT ↓85%
/tokens	3.3s 1.8s	510ms 190ms	LCP ↓45% · TBT ↓63%

- 최대 75행 테이블에 가상화를 적용했지만 화면에 공백이 생겨 되돌리고, 첫 화면의 행 수를 줄인 뒤 나머지를 단계적으로 렌더링
- 실시간으로 무거운 행이 계속 추가되던 테이블의 자동 추가량을 제한하고, 신규·대기 거래 알림을 모아 갱신
- Jest를 Vitest로 옮기고, 렌더링 변경 후 화면이 밀리거나 실시간 갱신이 누락되는지 Playwright 테스트로 확인

02 /

WebView 거래 화면과 파트너별 기능 분리

- **Pockie miniDApp** — BiFi·Swap·BTCFi 거래 화면을 개발하고, 모바일 앱·Chrome Extension의 지갑 기능을 postMessage로 연결
- **Pockie miniDApp** — 앱 재배포 없이 플랫폼·버전별로 기능을 켜고 끌 수 있도록 BFF config v2에 feature flag 구현
- **BTCFi Partners** — 상품 조건과 지갑 연동 방식이 달라 거래 흐름을 파트너별로 분리 — 파트너 수와 배포 일정을 고려해 모노레포 도입 대신 기존 저장소에서 경계를 나누고 CI로 의존 규칙 검사

03 /

사내 AI Agent 런타임 개선

- Claude 종량제 요금 변동에 대응해 Codex로 전환 — 오픈소스의 잦은 업데이트를 따라가는 부담도 고려해 App Server를 선택하고, 요청별 상태·취소·종료를 분리
- 쓰지 않는 MCP 서버까지 매번 시작하던 낭비를 줄이기 위해, 도구를 호출할 때 필요한 서버만 실행하는 lazy proxy 구현
- 요청을 읽기·쓰기만으로 나누던 권한 정책을 바꿔, 각 도구가 외부에 미치는 영향과 되돌릴 수 있는지를 보고 실행·승인·차단 결정
- 자동 리뷰가 저장소별 규칙을 따르도록 하고 리뷰 근거·confidence·CI 결과를 확인한 뒤 승인하도록 구성, 최종 merge는 사람이 판단

04 /

DApp 데이터 조회 구조 최적화

- **Gas Top-up** — SDK의 중복 자산 조회와 불필요한 vault 조회를 제거하고 갱신 주기를 조정해 최초 API 호출을 25~46회→17회, 주기 호출 분당 25회→1~19회로 축소
- **BiFi WebView** — 변경이 드문 bridge pair를 자산마다 반복 조회하던 구조를 최초 1회+pair별 N회에서 outbound·inbound 각 1회로 줄이고 React Query 캐시로 재사용

CERTIFICATIONS & ACTIVITIES

AWS Certified Solutions Architect – Associate (2026)
정보처리기사 (2019)
OSSCA — githru-vscode-ext (2023)
SmileGate Membership AI 1기 (2021)
OSSCA — TS Handbook 한글화 (2020)
AUSG (AWSKRUG University Student Group) 1기 (2019)

LANGUAGES

영어 — 기초 비즈니스 (Opic IM1)
일본어 — 기초 비즈니스 (JLPT N3)